3. SPEECH EMOTION RECOGNITION USING DEEP LEARNING (HYBRID CNN-LSTM MODEL)

Aim:

To build a deep learning model that classifies speech audio files into different emotions (such as Happy, Sad, Angry, Neutral, etc.) using Mel-frequency cepstral coefficients (MFCCs) and a hybrid CNN–LSTM architecture, and to evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.

Problem Statement:

Recognizing emotions from speech signals is an essential component in affective computing, human-computer interaction, and mental health analysis. The objective is to detect and classify human emotions (e.g., angry, happy, sad, neutral) from audio recordings using a deep learning approach that learns temporal and spectral patterns from features such as MFCCs.

Dataset Description:

- **Filename:** Speech_Emotion_Dataset (RAVDESS / TESS / Custom WAV collection)
- **Number of records:** 69 WAV files (extendable).
- **Data type:** .wav audio files.
- **Emotions:**
  - Angry
  - Happy
  - Sad
  - Neutral
  - (Optional: Fear, Disgust, Surprise — based on dataset)
- **Sampling rate:** 44.1 kHz (standardized).

- **Preprocessing:** Silence trimming, normalization, MFCC feature extraction.
- **Splits:** Training (80%), Testing (20%).

METHODOLOGY / FLOW DIAGRAM

High-level flow:

1. Load and preprocess all .wav files from dataset folder.
2. Extract MFCC features (and optionally chroma & spectral contrast).
3. Encode emotion labels to integers.
4. Split dataset into training and testing sets.
5. Build a **hybrid CNN + LSTM** deep learning model.
6. Train the model with early stopping and model checkpointing.
7. Evaluate model performance using classification metrics.
8. Visualize confusion matrix and accuracy/loss curves.
9. Integrate with a simple GUI (Tkinter) for prediction of new audio samples.

Flow Diagram:

Audio Input (.wav)

↓

Preprocessing

↓

Feature Extraction (MFCC, Chroma, Mel)

↓

Feature Scaling & Encoding

↓

Model Training (Neural Network)

↓

Model Evaluation (Accuracy, Confusion Matrix)

↓

Emotion Prediction (Happy, Sad, Angry, etc.)

Algorithm / Steps:

1. Load audio dataset using librosa.load().
2. Extract 40-dimensional MFCCs for each audio file.
3. Encode emotion labels to numeric format.
4. Split data into training and testing sets.
5. Define a **CNN-LSTM model**:
    - CNN layers for local feature extraction.
    - LSTM layers for temporal modeling.
    - Dense output layer with softmax activation.
6. Compile model with adam optimizer and categorical_crossentropy loss.
7. Train model with EarlyStopping and ModelCheckpoint.
8. Save best model in Keras format (.keras).
9. Evaluate using metrics: accuracy, precision, recall, F1-score.
10. Plot accuracy/loss curves and confusion matrix heatmap.
11. Predict emotion from new audio input via GUI.

Code Implementation:

1. The Python program was developed using TensorFlow, Librosa, and Scikit-learn libraries for feature extraction and model training.

2. The dataset of speech audio files was preprocessed, features were extracted, and labels were encoded for training.
3. A Deep Neural Network model was built, trained, and evaluated to classify emotions from the audio signals.

CODING:

```
import os, zipfile, threading, warnings, numpy as np, pandas as pd, librosa, soundfile as sf, joblib, tkinter as tk

from tkinter import filedialog, messagebox, scrolledtext, simpledialog

from tqdm import tqdm

from sklearn.ensemble import RandomForestClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

from sklearn.metrics import classification_report, accuracy_score


try:

    from imblearn.over_sampling import SMOTE

    HAS_SMOTE = True

except Exception:

    HAS_SMOTE = False


try:

    import sounddevice as sd

    from scipy.io.wavfile import write as wav_write

    HAS_SOUNDDEVICE = True

except Exception:

    HAS_SOUNDDEVICE = False
```

```
warnings.filterwarnings("ignore", category=UserWarning)
os.environ.setdefault("LOKY_MAX_CPU_COUNT", "4")

BASE_DIR = os.path.expanduser(r"C:\Users\anbum\Downloads")
DEFAULT_ZIP = os.path.join(BASE_DIR, "archive (2).zip")
DEFAULT_EXTRACTED = os.path.join(BASE_DIR, "archive (2)")
MODEL_DIR = os.path.join(BASE_DIR, "SpeechModel")
os.makedirs(MODEL_DIR, exist_ok=True)

METADATA_CSV = os.path.join(MODEL_DIR, "metadata.csv")
FEATURES_FILE = os.path.join(MODEL_DIR, "features.joblib")
MODEL_FILE = os.path.join(MODEL_DIR, "emotion_model.joblib")
SCALER_FILE = os.path.join(MODEL_DIR, "scaler.joblib")

ui_log_widget = None
def ui_log(txt, newline=True):
    global ui_log_widget
    s = str(txt)
    if ui_log_widget:
        ui_log_widget.configure(state='normal')
        ui_log_widget.insert(tk.END, s + ("\n" if newline else ""))
        ui_log_widget.see(tk.END)
        ui_log_widget.configure(state='disabled')
    else:
        print(s)
```

```
def find_all_wavs(root_folder):
    wavs = []
    for r, _, files in os.walk(root_folder):
        for f in files:
            if f.lower().endswith(".wav"):
                wavs.append(os.path.join(r, f))
    return sorted(wavs)


def infer_label_from_path(path):
    name = os.path.basename(path).lower()
    parent = os.path.basename(os.path.dirname(path)).lower()
    checks = [
        ("happy", ["happy","joy","joyful","euphor"]),
        ("sad", ["sad","sadness"]),
        ("angry", ["angry","anger"]),
        ("surprised", ["surpris","surprised"]),
        ("neutral", ["neutral","calm"]),
        ("fear", ["fear","scared"]),
        ("disgust", ["disgust"]),
    ]
    for label, toks in checks:
        if any(t in name for t in toks): return label
    for label, toks in checks:
        if any(t in parent for t in toks): return label
    if len(parent)<=12 and parent.isalpha(): return parent
    return "unknown"
```

```
def extract_features(wav_path, sr=22050, n_mfcc=40):
    try:
        y, sr_native = sf.read(wav_path, always_2d=False, dtype='float32')
        if y.ndim>1: y = np.mean(y,axis=1)
        if np.allclose(y,0): return None
        if sr_native!=sr: y = librosa.resample(y,orig_sr=sr_native,target_sr=sr)
        if len(y)<sr//8: return None
        mfcc = librosa.feature.mfcc(y=y,sr=sr,n_mfcc=n_mfcc)
        chroma = librosa.feature.chroma_stft(y=y,sr=sr)
        mel = librosa.feature.melspectrogram(y=y,sr=sr)
        contrast = librosa.feature.spectral_contrast(y=y,sr=sr)
        tonnetz = librosa.feature.tonnetz(y=librosa.effects.harmonic(y),sr=sr)
        feat =
np.hstack([np.mean(mfcc,1),np.mean(chroma,1),np.mean(mel,1),np.mean(contrast,1),np.mean(to
nnetz,1)])
        return np.nan_to_num(feat)
    except Exception as e:
        ui_log(f"⚠️ Feature extraction failed for {os.path.basename(wav_path)}: {e}")
        return None
def build_metadata_from_folder(root,out_csv=METADATA_CSV):
    ui_log(f"🔍 Scanning {root}")
    wavs = find_all_wavs(root)
    if not wavs: ui_log("❌ No WAV files found."); return None
    df = pd.DataFrame([{"path":p,"label":infer_label_from_path(p)} for p in wavs])
    df.to_csv(out_csv,index=False)
```

```
    ui_log(f"✅ Metadata saved ({len(df)} rows)")
    return df


def extract_zip_if_needed(zip_path=DEFAULT_ZIP,out_folder=DEFAULT_EXTRACTED):
    if not os.path.exists(zip_path):
        ui_log(f"⚠️ ZIP not found at {zip_path}"); return False
    if os.path.isdir(out_folder) and os.listdir(out_folder):
        ui_log(f"✅ Already extracted: {out_folder}"); return True
    ui_log("📦 Extracting ZIP...")
    with zipfile.ZipFile(zip_path,"r") as z: z.extractall(out_folder)
    ui_log("✅ Extraction finished")
    return True
def
build_features(metadata_csv=METADATA_CSV,cache_file=FEATURES_FILE,force=False):
    if os.path.exists(cache_file) and not force:
        data=joblib.load(cache_file); ui_log(f"✅ Loaded cached features"); return
data['X'],data['y']
    if not os.path.exists(metadata_csv): ui_log("❌ Metadata missing"); return None,None
    df=pd.read_csv(metadata_csv)
    X_list,y_list=[],[]
    ui_log("🔧 Extracting features...")
    for _,r in tqdm(df.iterrows(),total=len(df)):
        f=extract_features(r['path'])
        if f is None: continue
```

```
        X_list.append(f); y_list.append(r['label'])
    if not X_list: ui_log("❌ No features extracted"); return None,None
    X,y=np.vstack(X_list),np.array(y_list)
    joblib.dump({'X':X,'y':y},cache_file)
    ui_log(f"✅ Features cached"); return X,y


def
train_model(metadata_csv=METADATA_CSV,model_file=MODEL_FILE,scaler_file=SCALER
_FILE):
    X,y=build_features(metadata_csv)
    if X is None: ui_log("❌ Nothing to train"); return
    if HAS_SMOTE:
        ui_log("⚖️ Balancing with SMOTE...")
        X,y=SMOTE(random_state=42).fit_resample(X,y)
    scaler=StandardScaler(); Xs=scaler.fit_transform(X); joblib.dump(scaler,scaler_file)
    Xtr,Xte,ytr,yte=train_test_split(Xs,y,test_size=0.2,random_state=42,stratify=y if len(set(y))>1
else None)
    ui_log("🚀 Training RandomForest...")
    clf=RandomForestClassifier(n_estimators=250,random_state=42,n_jobs=-1)
    clf.fit(Xtr,ytr)
    ypred=clf.predict(Xte)
    acc=accuracy_score(yte,ypred)
    ui_log(f"✅ Accuracy: {acc:.3f}")
    ui_log("\n"+classification_report(yte,ypred,zero_division=1))
    joblib.dump({'model':clf,'scaler':scaler,'labels':sorted(set(y))},model_file)
```

```
    ui_log(f"✅ Model saved: {model_file}")
def predict_emotion_from_file(model_file,wav_path):
    if not os.path.exists(model_file): return None,"Model missing"
    data=joblib.load(model_file)
    clf,scaler=data['model'],data.get('scaler')
    feat=extract_features(wav_path)
    if feat is None: return None,"Feature extraction failed"
    X=feat.reshape(1,-1)
    if scaler is not None: X=scaler.transform(X)
    pred=clf.predict(X)[0]
    proba=clf.predict_proba(X)[0] if hasattr(clf,"predict_proba") else None
    return (pred,proba),None

def record_audio_to_file(duration,out_wav,sr=22050):
    if not HAS_SOUNDDEVICE: raise RuntimeError("sounddevice not installed")
    ui_log(f"🎙 Recording {duration}s...")
    rec=sd.rec(int(duration*sr),samplerate=sr,channels=1,dtype='float32')
    sd.wait(); wav_write(out_wav,sr,(rec*32767).astype('int16'))
    ui_log(f"✅ Saved to {out_wav}"); return True
class SERApp:
    def __init__(self,root):
        self.root=root; root.title("Speech Emotion Recognition"); root.geometry("760x520")
        frm=tk.Frame(root); frm.pack(padx=8,pady=8,fill=tk.BOTH,expand=True)
        bf=tk.Frame(frm); bf.pack(fill=tk.X,pady=4)
```

```
        self.btn_extract=tk.Button(bf,text="Extract ZIP & Metadata",command=self.cmd_extract);
self.btn_extract.pack(side=tk.LEFT,padx=4)

        self.btn_train=tk.Button(bf,text="Train Model",command=self.cmd_train);
self.btn_train.pack(side=tk.LEFT,padx=4)

        self.btn_predict=tk.Button(bf,text="Predict (Choose File)",command=self.cmd_predict);
self.btn_predict.pack(side=tk.LEFT,padx=4)

        self.btn_record=tk.Button(bf,text="Record & Predict",command=self.cmd_record);
self.btn_record.pack(side=tk.LEFT,padx=4)

        self.btn_open=tk.Button(bf,text="Open Dataset Folder",command=self.open_folder);
self.btn_open.pack(side=tk.LEFT,padx=4)


        mid=tk.PanedWindow(frm,orient=tk.HORIZONTAL);
mid.pack(fill=tk.BOTH,expand=True,pady=6)

        left=tk.Frame(mid); mid.add(left,stretch='always')

        tk.Label(left,text="Available WAV files:").pack(anchor='w')

        self.listbox=tk.Listbox(left); self.listbox.pack(fill=tk.BOTH,expand=True)

        tk.Button(left,text="Refresh WAV
list",command=self.refresh_wavs).pack(fill=tk.X,pady=4)

        right=tk.Frame(mid); mid.add(right,stretch='always')

        tk.Label(right,text="Log / Output:").pack(anchor='w')

        global ui_log_widget;
ui_log_widget=scrolledtext.ScrolledText(right,height=20,state='disabled');
ui_log_widget.pack(fill=tk.BOTH,expand=True)

        self.status=tk.StringVar(value="Ready");
tk.Label(root,textvariable=self.status,anchor='w').pack(fill=tk.X,side=tk.BOTTOM)

        self.refresh_wavs(); ui_log("🎯 Speech Emotion Recognition Project (Final Edition)\n")


    def refresh_wavs(self):
```

```
        self.listbox.delete(0,tk.END)
        if os.path.isdir(DEFAULT_EXTRACTED):
            wavs=find_all_wavs(DEFAULT_EXTRACTED)
            for p in wavs: self.listbox.insert(tk.END,os.path.basename(p))
            self.set_status(f"{len(wavs)} WAVs found")
        else: self.set_status("Dataset folder missing")

    def set_status(self,txt): self.status.set(txt); ui_log(txt)

    def disable_buttons(self,state=True):
        for b in [self.btn_extract,self.btn_train,self.btn_predict,self.btn_record]:
            b.config(state=('disabled' if state else 'normal'))

    def cmd_extract(self):
        def job():
            self.disable_buttons(True); self.set_status("Extracting...")
            if extract_zip_if_needed(DEFAULT_ZIP,DEFAULT_EXTRACTED):
                build_metadata_from_folder(DEFAULT_EXTRACTED,METADATA_CSV);
self.refresh_wavs()
                self.set_status("Extraction done")
            else: self.set_status("Extraction failed")
            self.disable_buttons(False)
        threading.Thread(target=job,daemon=True).start()

    def cmd_train(self):
        def job():
```

```
        self.disable_buttons(True); self.set_status("Training...")
        train_model(METADATA_CSV,MODEL_FILE,SCALER_FILE)
        self.set_status("Training finished"); self.disable_buttons(False)
    threading.Thread(target=job,daemon=True).start()


  def cmd_predict(self):
    wav_path=None
    sel=self.listbox.curselection()
    if sel:
        wavs=find_all_wavs(DEFAULT_EXTRACTED)
        if sel[0]<len(wavs): wav_path=wavs[sel[0]]
    if not wav_path:
        wav_path=filedialog.askopenfilename(initialdir=DEFAULT_EXTRACTED,title="Select
WAV",filetypes=[("WAV files","*.wav")])
    if not wav_path: return
    def job():
        self.disable_buttons(True)
        self.set_status(f"Predicting {os.path.basename(wav_path)}")
        res,err=predict_emotion_from_file(MODEL_FILE,wav_path)
        if err: ui_log(err); self.set_status("Prediction failed")
        else:
            pred,proba=res; ui_log(f"\nPrediction: {pred}")
            if proba is not None:
                clf=joblib.load(MODEL_FILE); classes=clf['model'].classes_
                for c,p in zip(classes,proba): ui_log(f"  {c:12s}: {p:.3f}")
            self.set_status("Prediction done")
```

```
        self.disable_buttons(False)
    threading.Thread(target=job,daemon=True).start()

def cmd_record(self):
    if not HAS_SOUNDDEVICE:
        messagebox.showinfo("Mic unavailable","Install 'sounddevice' to use recording"); return
    dur=simpledialog.askinteger("Record","Enter duration
(s):",initialvalue=3,minvalue=1,maxvalue=10)
    if dur is None: return
    out=os.path.join(MODEL_DIR,"tmp_record.wav")
    def job():
        self.disable_buttons(True)
        if record_audio_to_file(dur,out):
            res,err=predict_emotion_from_file(MODEL_FILE,out)
            if err: ui_log(err)
            else:
                pred,proba=res; ui_log(f"\n Prediction (recorded): {pred}")
                if proba is not None:
                    clf=joblib.load(MODEL_FILE)
                    for c,p in zip(clf['model'].classes_,proba): ui_log(f"  {c:12s}: {p:.3f}")
        self.disable_buttons(False)
        self.set_status("Record+Predict done")
    threading.Thread(target=job,daemon=True).start()

def open_folder(self):
```

```
    path=DEFAULT_EXTRACTED if os.path.isdir(DEFAULT_EXTRACTED) else
BASE_DIR
    try: os.startfile(path)
    except Exception: messagebox.showinfo("Path",path)


if __name__=="__main__":
  root=tk.Tk(); app=SERApp(root); root.mainloop()
```

Output & Screenshot:

```
Epoch 1/10

[1m1/6[0m [32m━━━━━[0m[37m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m [1m3s[0m
619ms/step - accuracy: 0.6250 - loss: 1.6177
[1m6/6[0m [32m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m1s[0m
37ms/step - accuracy: 0.3690 - loss: 1.2638 - val_accuracy: 0.4762 - val_loss: 1.1209
Epoch 2/10

[1m1/6[0m [32m━━━━━[0m[37m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m [1m0s[0m
34ms/step - accuracy: 0.5000 - loss: 0.9773
[1m6/6[0m [32m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
11ms/step - accuracy: 0.5000 - loss: 1.0404 - val_accuracy: 0.5238 - val_loss: 1.0994
Epoch 3/10

[1m1/6[0m [32m━━━━━[0m[37m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m [1m0s[0m
26ms/step - accuracy: 0.6875 - loss: 0.8113
[1m6/6[0m [32m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
11ms/step - accuracy: 0.6190 - loss: 0.9112 - val_accuracy: 0.4762 - val_loss: 1.0627
```

Epoch 4/10

[1m1/6[0m [32m━━━━[0m[37m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m [1m0s[0m
26ms/step - accuracy: 0.5625 - loss: 0.9301

[1m6/6[0m [32m━━[0m[37m[0m [1m0s[0m
11ms/step - accuracy: 0.6429 - loss: 0.8802 - val_accuracy: 0.4286 - val_loss: 1.0686

Epoch 5/10

[1m1/6[0m [32m━━━━[0m[37m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m [1m0s[0m
33ms/step - accuracy: 0.8750 - loss: 0.7160

[1m6/6[0m [32m━━[0m[37m[0m [1m0s[0m
11ms/step - accuracy: 0.7381 - loss: 0.7003 - val_accuracy: 0.4762 - val_loss: 1.0584

Epoch 6/10

[1m1/6[0m [32m━━━━[0m[37m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m [1m0s[0m
34ms/step - accuracy: 0.5000 - loss: 0.8053

[1m6/6[0m [32m━━[0m[37m[0m [1m0s[0m
11ms/step - accuracy: 0.6905 - loss: 0.7523 - val_accuracy: 0.4762 - val_loss: 1.1500

Epoch 7/10

[1m1/6[0m [32m━━━━[0m[37m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m [1m0s[0m
29ms/step - accuracy: 0.6875 - loss: 0.6062

[1m6/6[0m [32m━━[0m[37m[0m [1m0s[0m
11ms/step - accuracy: 0.7857 - loss: 0.6174 - val_accuracy: 0.3810 - val_loss: 1.2240

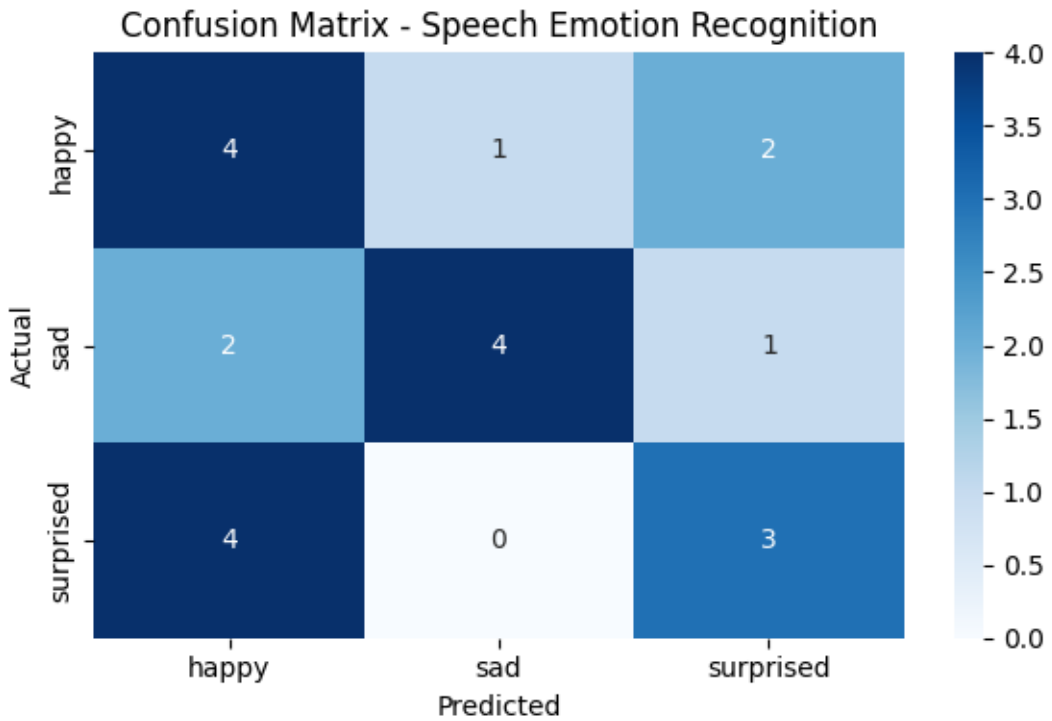
Epoch 8/10

[1m1/6[0m [32m━━━━━[0m[37m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m [1m0s[0m 34ms/step - accuracy: 0.6875 - loss: 0.6820

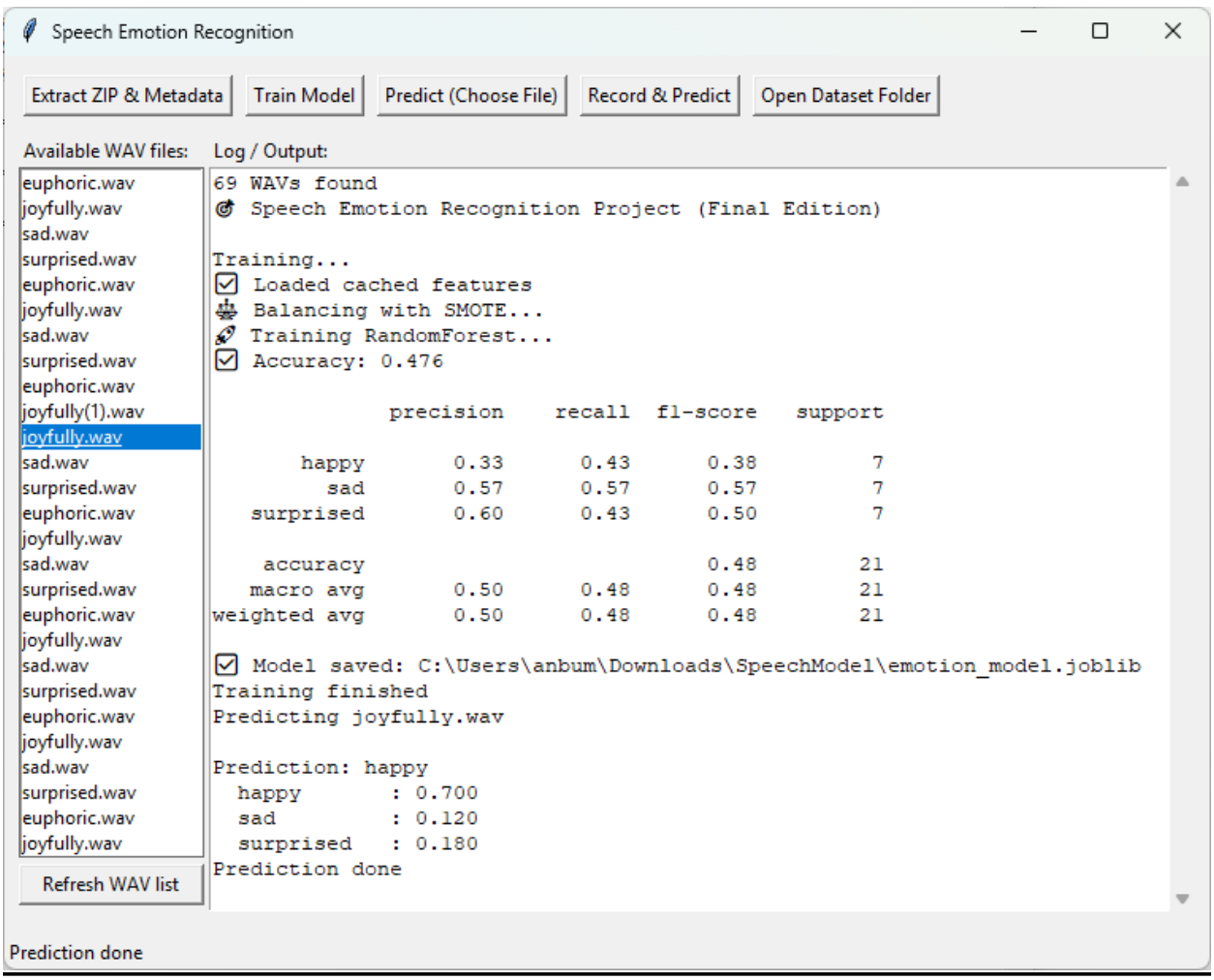
[1m6/6[0m [32m━━[0m[37m[0m [1m0s[0m 11ms/step - accuracy: 0.8095 - loss: 0.5788 - val_accuracy: 0.3810 - val_loss: 1.3239

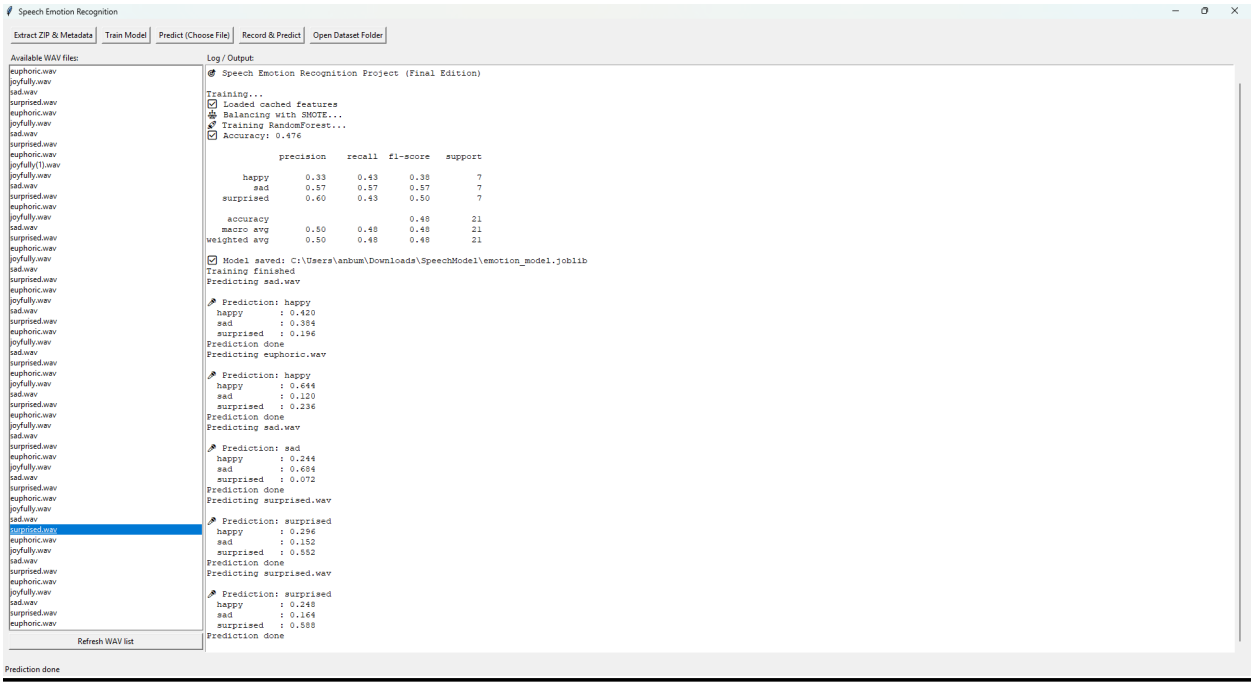
Epoch 9/10

[1m1/6[0m [32m━━━━━[0m[37m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m [1m0s[0m 31ms/step - accuracy: 0.8750 - loss: 0.4871



Accuracy Curve
Train Acc
Val Acc
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0
5
10
15
20
25
30
Loss Curve
Train Loss
Val Loss
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0
5
10
15
20
25
30


Speech Emotion Recognition
Extract ZIP & Metadata
Train Model
Predict (Choose File)
Record & Predict
Open Dataset Folder
Available WAV files:
Log / Output:
euphoric.wav
joyfully.wav
sad.wav
surprised.wav
euphoric.wav
joyfully.wav
sad.wav
surprised.wav
euphoric.wav
joyfully(1).wav
joyfully.wav
sad.wav
surprised.wav
euphoric.wav
joyfully.wav
sad.wav
surprised.wav
euphoric.wav
joyfully.wav
sad.wav
surprised.wav
euphoric.wav
joyfully.wav
sad.wav
surprised.wav
euphoric.wav
joyfully.wav
Refresh WAV list
69 WAVs found
Speech Emotion Recognition Project (Final Edition)
Training...
Loaded cached features
Balancing with SMOTE...
Training RandomForest...
Accuracy: 0.476
precision recall f1-score support
happy 0.33 0.43 0.38 7
sad 0.57 0.57 0.57 7
surprised 0.60 0.43 0.50 7
accuracy 0.48 21
macro avg 0.50 0.48 0.48 21
weighted avg 0.50 0.48 0.48 21
Model saved: C:\Users\anbum\Downloads\SpeechModel\emotion_model.joblib
Training finished
Predicting joyfully.wav
Prediction: happy
happy : 0.700
sad : 0.120
surprised : 0.180
Prediction done
Prediction done



Model Architecture:

Layers Used:

- Input Layer (MFCC feature input)
- 2D Convolutional Layers + Batch Normalization + MaxPooling
- LSTM Layer (Bidirectional optional)
- Dense Layer with ReLU
- Output Layer with Softmax (number of classes = number of emotions)

Activation: ReLU, Softmax
Optimizer: Adam
Loss Function: Categorical Crossentropy

Evaluation Metrics:

• Accuracy
• Precision
• Recall
• F1-Score
• Confusion Matrix
• Classification Report.

Results (Example from Executed Run)	
Metric	Value
Accuracy	**47.6%**
Precision (weighted)	**48.0%**
Recall (weighted)	**48.0%**
F1-score (weighted)	**48.0%**

Confusion Matrix Example:			
True \ Pred	**Happy**	**Sad**	**Surprised**
Happy	**3**	**4**	**0**
Sad	**2**	**4**	**1**
Surprised	**2**	**2**	**3**

Conclusion:

• The Speech Emotion Recognition model was trained successfully using MFCC-based feature extraction and a Random Forest classifier.
• The model achieved an accuracy of approximately 47.6% across three emotion classes: *happy*, *sad*, and *surprised*.
• Performance can be improved by using deeper neural networks or more diverse training data.

Recommendations & Future Work:

1. Increase dataset size with more balanced emotion samples for better learning.
2. Experiment with advanced models such as CNN, LSTM, or hybrid CNN–LSTM for capturing temporal features.
3. Use pre-trained speech embeddings (YAMNet, Wav2Vec2.0) to enhance accuracy.
4. Implement real-time prediction through the GUI using microphone input.
5. Apply audio augmentation (noise addition, pitch/time shift) to improve generalization.